

Camada de transporte

Aulas do livro disponíveis em:

<http://www-net.cs.umass.edu/kurose-ross-ppt-7e/>



Camada de transporte

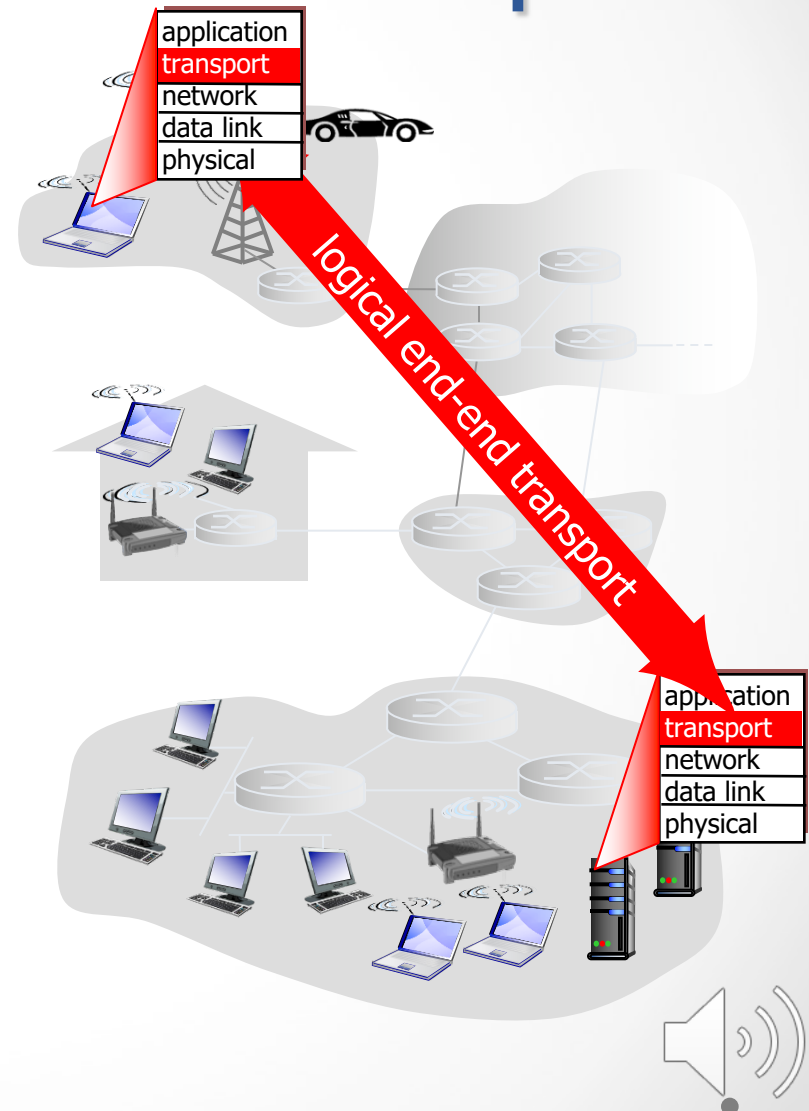
Objetivos:

- Entender os princípios por trás dos serviços da camada de transporte:
 - multiplexação, demultiplexação
 - transferência de dados confiável
 - controle de fluxo
 - controle de congestionamento
- aprender sobre os protocolos da camada de transporte Internet:
 - UDP: transporte sem conexão
 - TCP: Transporte confiável orientado a conexão
 - controle de congestionamento TCP



Protocolos e serviços de transporte

- fornecer **comunicação lógica** entre processos de aplicações que funcionam em diferentes hospedeiros
- protocolos de transporte são executados nos sistemas finais
 - lado envio: quebra as mensagens de aplicação em **segmentos**, passa para a camada de rede
 - lado receptor: segmentos reagrupados em mensagens, passa a camada de aplicação
- mais do que um protocolo de transporte disponível para aplicações
 - Internet: TCP e UDP



Camada de transporte vs. rede

- *Camada de rede:*
comunicação lógica entre máquinas
- *Camada de transporte:*
comunicação lógica entre processos
 - Melhora serviços da camada de rede

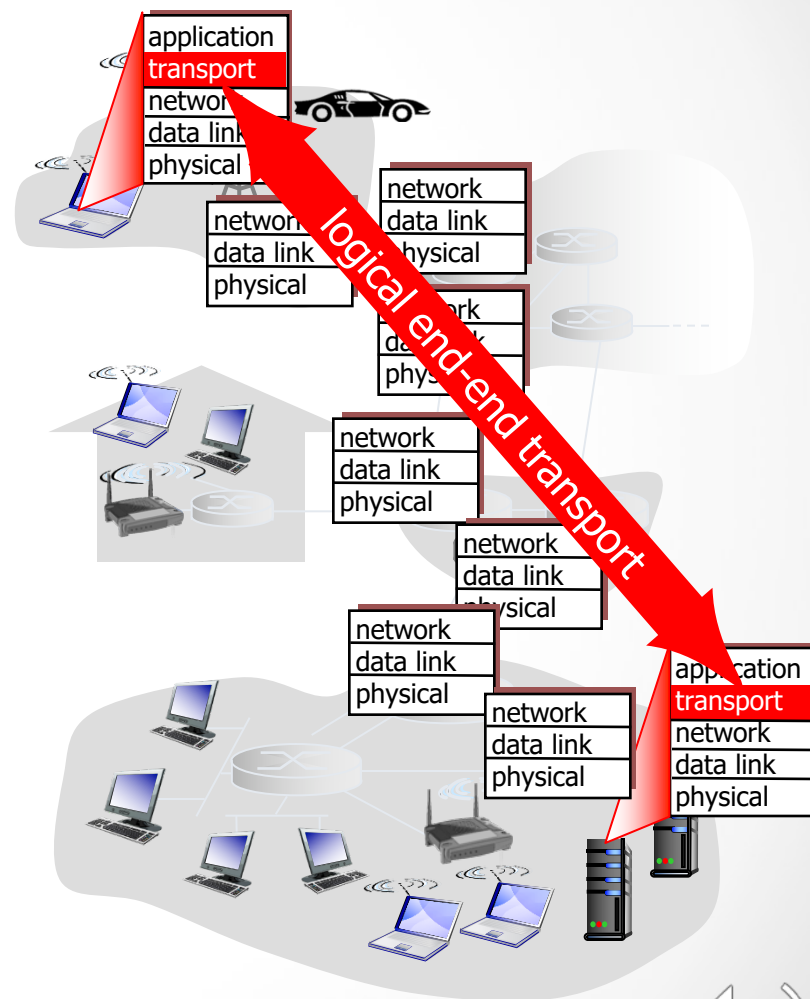
analogia:

- 12 crianças da casa da Ana enviam cartas para 12 crianças da casa do Pedro:*
- Máquinas (hosts) = casas
 - processos = crianças
 - Mensagens app = cartas em envelopes
 - protocolo de transporte = Ana e Pedro que demux aos irmãos em casa
 - protocolo de camada de rede = serviço postal



Protocolos de camada de transporte da Internet

- confiável, entrega em ordem (TCP)
 - controle de congestionamento
 - controle de fluxo
 - estabelecimento da conexão
- não confiável, entrega não ordenada: UDP
 - extensão do "melhor esforço" IP
- serviços não disponíveis:
 - Garantia de atraso
 - Garantia de banda passante



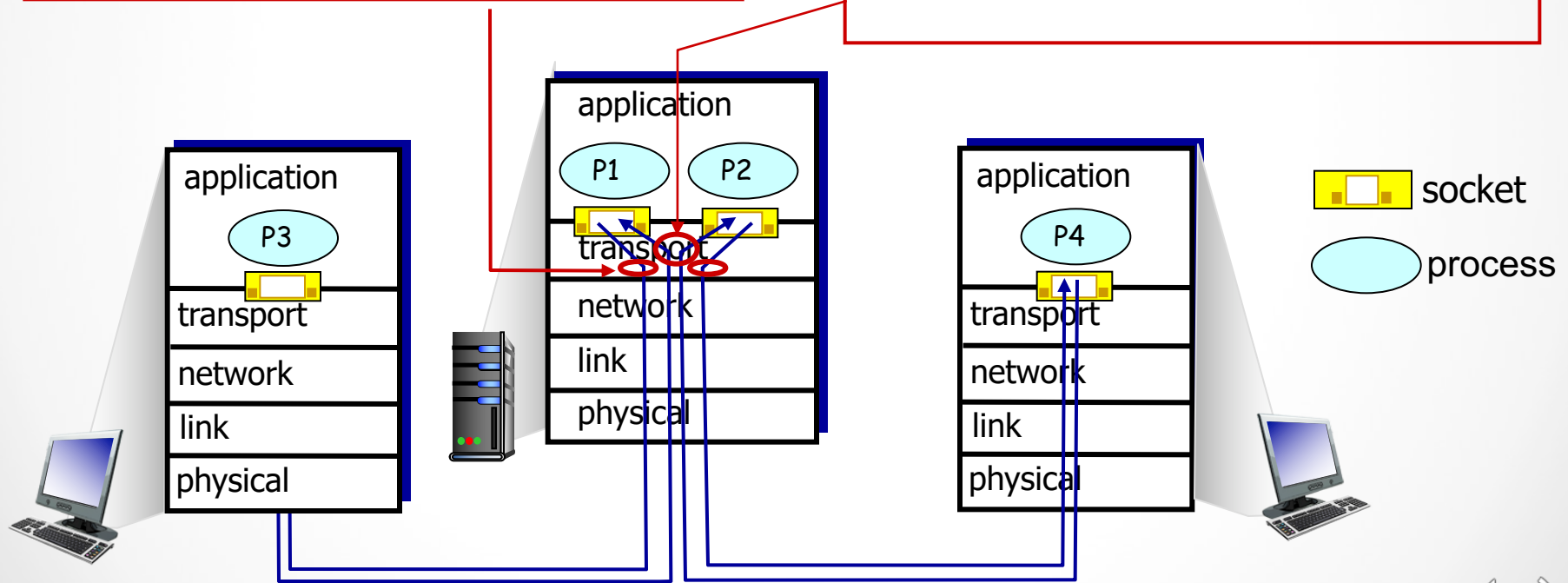
Multiplexação/demultiplexação

multiplexação no emissor

lidar com dados de múltiplos sockets, adicionar cabeçalho de transporte (mais tarde usado para demultiplexação)

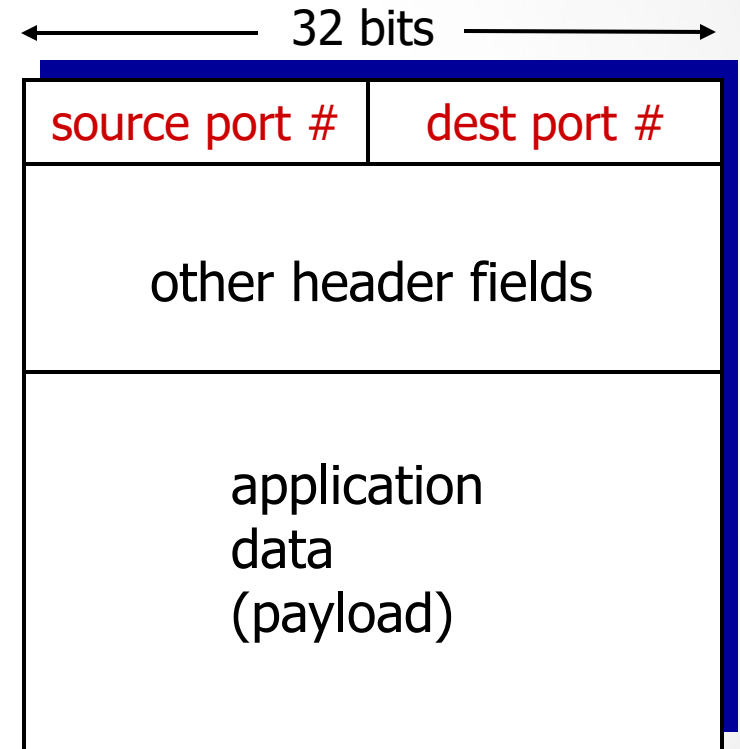
demultiplexação no receptor:

utilizar informações de cabeçalho de transporte para entregar os segmentos recebidos ao socket adequado



Como demultiplexação funciona

- Host recebe datagramas IP
 - Cada datagrama possui endereço IP de origem, endereço IP de destino
 - Cada datagrama carrega um segmento da camada de transporte
 - Cada segmento tem origem, número de porta de destino
- usa **endereços IP e números de porta** para enviar segmento para o socket apropriado



TCP/UDP segment format



Demultiplexação sem conexão

- Lembrar: o socket criado tem porta no host #:
`DatagramSocket mySocket1`
`new DatagramSocket(12534);`
- lembrar: Ao criar datagrama para enviar ao soquete UDP, deve especificar
 - Endereço IP destino
 - Porta destino #

-
- Quando um host recebe um segmento UDP:

- verifica porta de destino no segmento #
- segmento UDP direciona para socket com essa porta #



Datagramas IP com mesma *porta destino #*, mas diferentes *endereços IP de origem* e/ou *números de porta origem* serão direcionados para o mesmo socket no destino

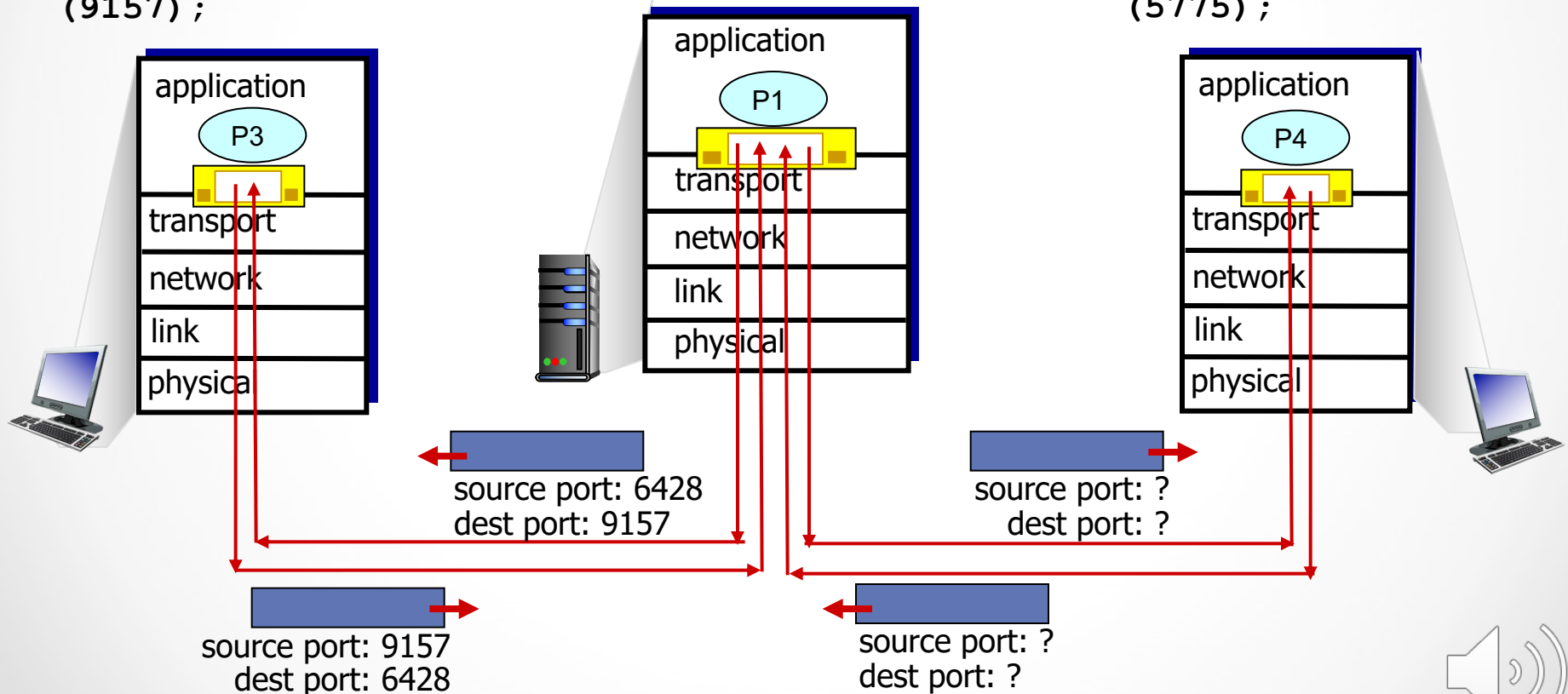


Demux sem conexão: exemplo

```
DatagramSocket  
mySocket2 = new  
DatagramSocket  
(9157) ;
```

```
DatagramSocket  
serverSocket = new  
DatagramSocket  
(6428) ;
```

```
DatagramSocket  
mySocket1 = new  
DatagramSocket  
(5775) ;
```

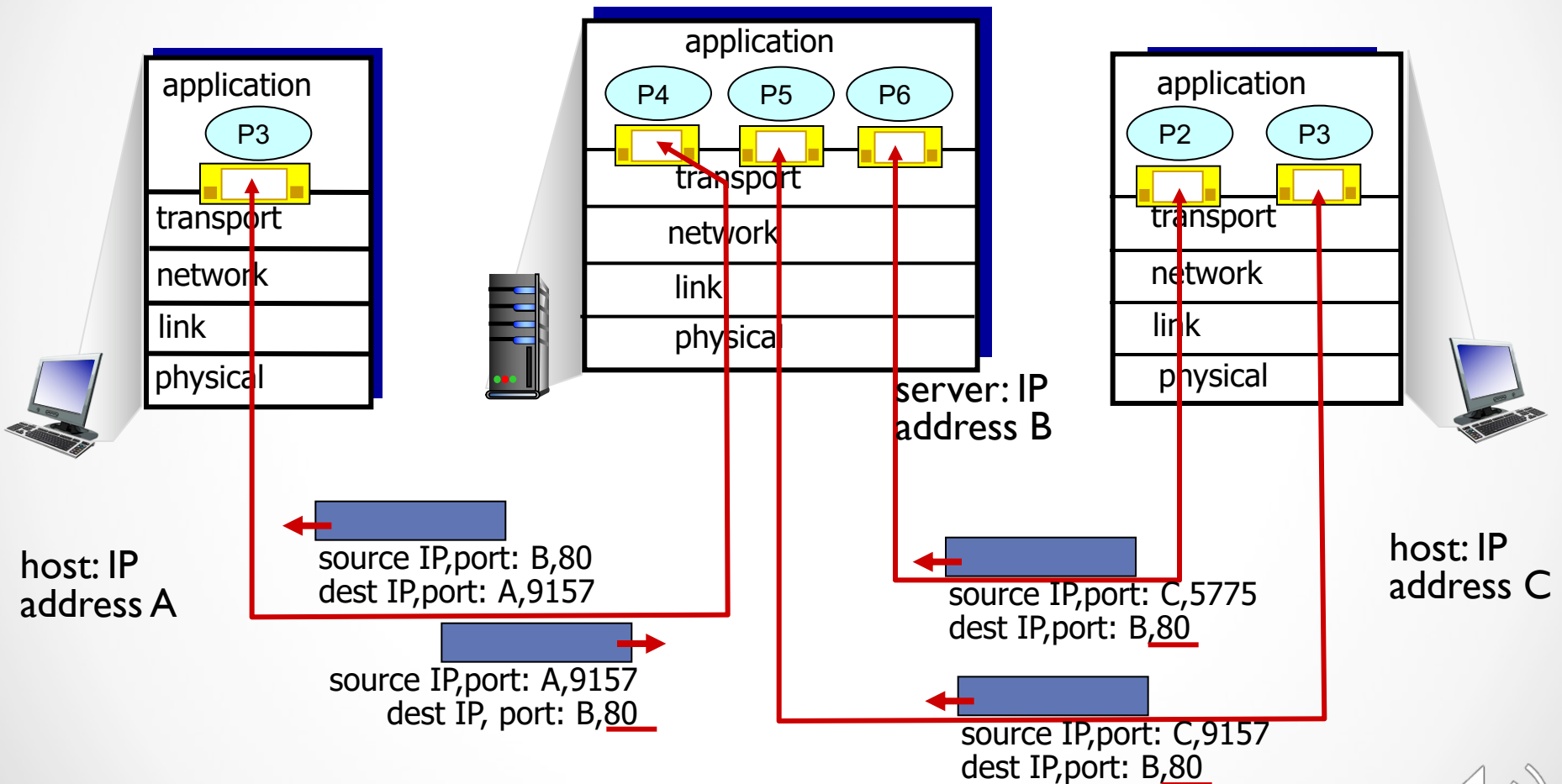


Demux orientado a conexão

- Socket TCP é identificado pela quadrupla (4-tuple):
 - Endereço IP origem (source IP address)
 - Numero de porta origem (source port number)
 - Endereço IP destino (dest IP address)
 - Numero de porta destino (dest port number)
- demux: Receptor usa todos os quatro valores para direcionar o segmento para o socket apropriado
- Host do servidor pode suportar muitos sockets TCP simultâneos:
 - Cada soquete identificado por sua própria quadrupla
- Os servidores web têm soquetes diferentes para cada cliente de conexão
 - HTTP não persistente terá socket diferente para cada solicitação



Demux orientado a conexão: exemplo

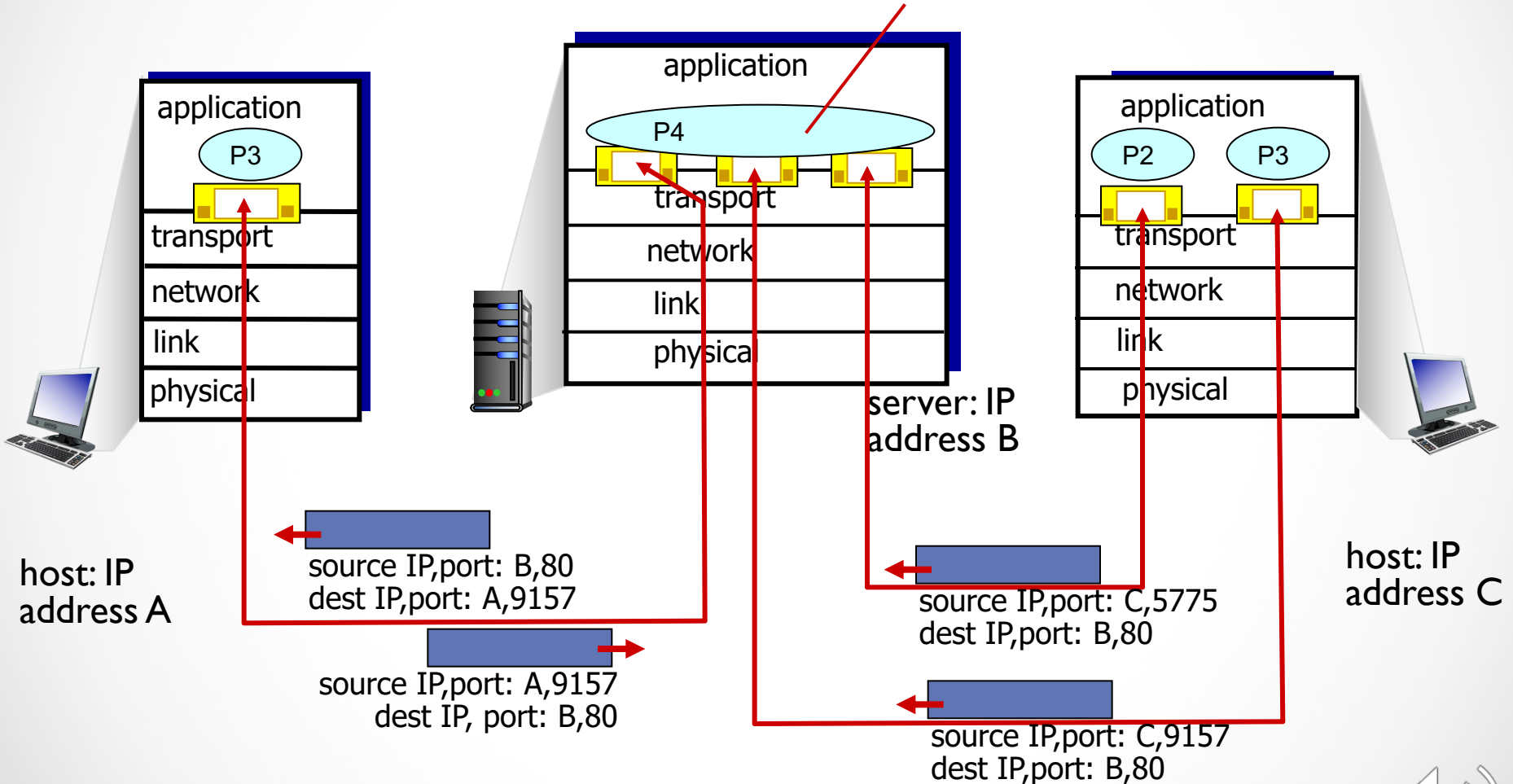


Três segmentos, todos destinados para o end IP: B,
porta dest: 80 são demultiplexadas em diferentes sockets



Demux orientado a conexão: exemplo

servidor multi thread

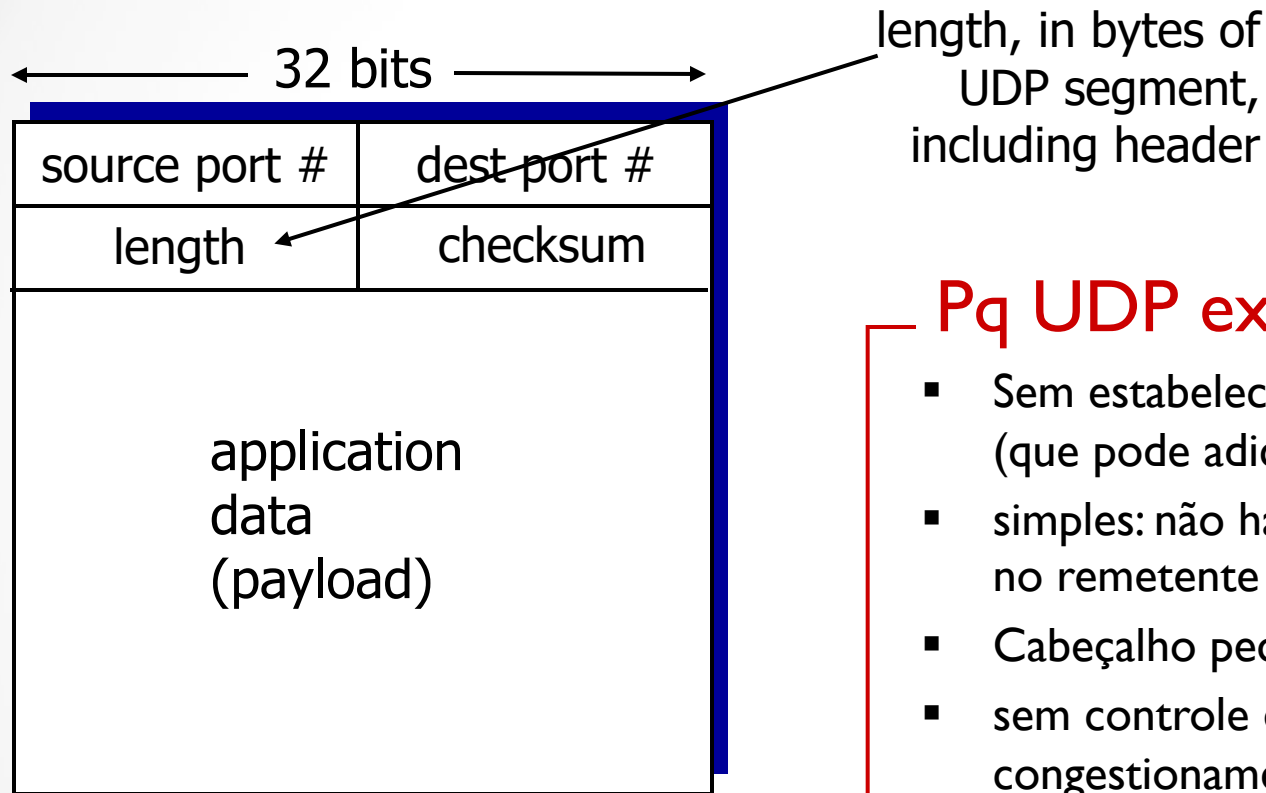


UDP: User Datagram Protocol [RFC 768]

- "Sem frescura", protocolo de transporte da Internet
- Serviço "melhor esforço, segmentos UDP podem ser:
 - perdidos
 - Entregues fora de ordem para a aplicação
- Sem conexão:
 - Sem handshaking entre origem e destino UDP
 - Cada segmento UDP é tratado independentemente dos outros
- UDP usado:
 - aplicativos de streaming multimídia (tolerante a perda, taxa sensível)
 - DNS
 - SNMP
- transferência confiável com UDP:
 - acrescentar confiabilidade na camada de aplicação
 - aplicação específica para recuperação de erros!



UDP: segment header



UDP segment format

Pq UDP existe?

- Sem estabelecimento de conexão (que pode adicionar atraso)
- simples: não há estado de conexão no remetente ou receptor
- Cabeçalho pequeno tamanho
- sem controle de congestionamento: UDP pode transmitir tão rápido quanto desejado



UDP checksum

Objetivo: detectar “erros” (e.g., bits trocados) no segmento transmitido

Remetente:

- tratar o conteúdo do segmento, incluindo os campos de cabeçalho, como sequência de números inteiros de 16-bits
- checksum: adição (uma soma de complemento) do conteúdo do segmento
- transmissor coloca o valor do checksum no campo UDP checksum

Receptor:

- calcular soma de verificação do segmento recebido
- verifica se o checksum calculado é igual ao valor do campo checksum:
- NÃO - erro detectado
- SIM - nenhum erro detectado. Talvez mais tarde



Checksum Internet: exemplo

exemplo: adicionar dois inteiros de 16 bits

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Nota: Quando adição de números, um valor a partir do bit mais significativo precisa ser adicionado ao resultado

* Confira os exercícios interativos on-line para mais exemplos: http://gaia.cs.umass.edu/kurose_ross/interactive/

